Umwelt-Campus Birkenfeld | HOCHSCHULE TRIER

Shrouded Dales

Projektbericht

Spieleentwicklung SS 2023
Dr. Markus Schwinn

Entwickler:

Göttel, Philipp: Matr.Nr. 971994

Kehrbach, Christina: Matr.Nr. 974669

Wittmann, Florian: Matr.Nr. 974743

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Eidesstattliche Erklärung 3
Einleitung 4
Planung 5
Grundidee 5
Leveldesign 5
Steuerung 6
Umsetzung 8
Leveldesign 8
Tilemap und Audiodesign 8
Main Screen 10
Menüs 10
Audiocontroller 11
State Machines 12
Player State Machine 12
Enemy State Machine 14
Inventory 14
Inventory Interface 14
Inventory Slots 15
Hotbar 15
Equipment Inventory 16
Chest 16
Item System 17
Pickups 17
Skills 17
Resource Interface 18
Probleme und Lerneffekte 18
State Machines 18
Audiocontroller 19
Quellenangaben für Assets 20
Soundscape: 20
Grafik: 21
Tilesets 21
Sprites 21

Eidesstattliche Erklärung

Hiermit versichern wir, dass wir die Hausarbeit selbstständig verfasst haben und keine anderen, als die angegebenen Quellen und Hilfsmittel benutzt haben, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

06.10.23 Philipp Göttel

Datum, Unterschrift Philipp Göttel

06.10.23 Christina Kehrbach

Datum, Unterschrift Christina Kehrbach

06.10.2023, Florian Wittmann

Datum, Unterschrift Florian Wittmann

Einleitung

"It's like in the great stories, Mr. Frodo.
The ones that really mattered.
Full of darkness and danger they were.
And sometimes you didn't want to know the end…
because how could the end be happy?
How could the world go back to the way it was,
when so much bad had happened?
But in the end, it's only a passing thing… this shadow.
Even darkness must pass..."
~ JRR Tolkien

Hinter einer hohen Bergkette liegen in einem dicht bewaldeten Hügelland liegen vereinzelte Siedlungen. Die Menschen hier sind hartgesotten und führen ein einfaches Leben.

Für "Cirilla", ein Mitglied der Wache, beginnt hier eine unerwartete Reise.

Auf einer Routine-Patrouille einen der Pfade entlang, die wie Lebensadern die Siedlungen verbinden, entdeckt sie eine Gruppe Untoter, im dichten Unterholz verborgen.

Mutig nimmt sie die Verfolgung auf, woraufhin die Eindringlinge sich tiefer in das Herz des Waldes zurückziehen.

Ein Portal in einem verwachsenen Hain scheint der Ursprung der Untoten zu sein, doch als sie sich zurückziehen will, muss sie feststellen, dass sie umringt wurde und ein chaotischer Kampf entbrennt, der für sie damit endet, dass sie in das Portal gerät.

Kann sie ihren Weg durch die unbekannte Umgebung nach Hause finden? Was steckt hinter der Ausbreitung der Untoten und anderer Monster und lässt es sich verhindern?

Planung

Grundidee

Ursprünglich haben wir an ein futuristisches Setting gedacht, in dem der Spieler zahlreiche Planetenoberflächen oder Planetoiden besuchen kann, um auf diesen Rohstoffe abzubauen.

Aufgrund der hohen Verfügbarkeit von Fantasy-Assets und der eher spärlichen Auswahl an guten SciFi-Assets haben wir diesen Ansatz in einem Team aus 3 Entwicklern verworfen. Eine einzelne Person nur für das Erstellen von Grafiken abzustellen, hätte uns zu viel Entwickler-Zeit gekostet.

Die Grundidee blieb bestehen:

Wir wollten ein Spiel entwickeln, bei dem der Charakter immer wieder in die verschiedensten Gebiete vordringt, zwischen diesen Leveln jedoch in ein sich dynamisch mit Spiel-Progress entwickelndes Hub-Level(eine Siedlung) mit Quest- und Händler-NPCs besuchen kann.

Der Spieler sollte sowohl selbst Ressourcen von bestimmten Objekten abbauen können, als auch Item-Drops von besiegten Gegnern aufsammeln können, um diese im Hub gegen Währung oder andere Items zu tauschen.

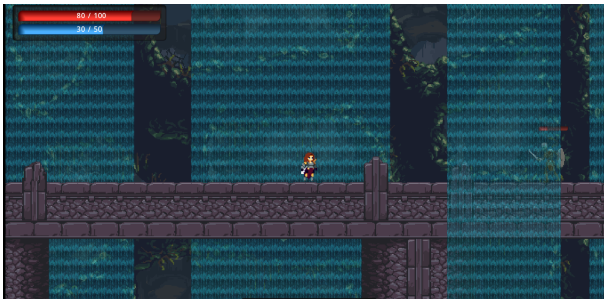
Darüber hinaus soll es ein Charaktermenü geben, in dem der Spieler mit wachsender Erfahrung des Charakters Skillpunkte verteilen kann.

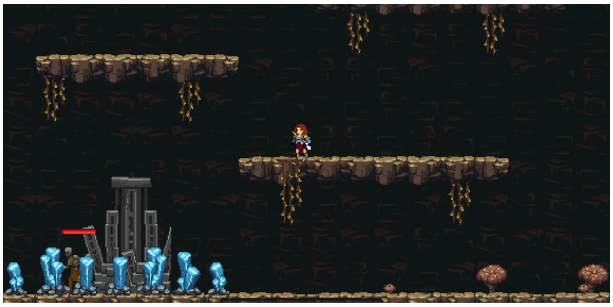
Leveldesign

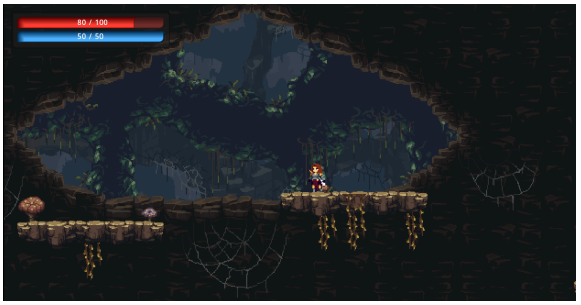
Wir haben uns zu Beginn relativ viele Gedanken um das Leveldesign gemacht und in Richtung prozedural-generiertes Gelände Nachforschungen angestellt. Aus Zeitgründen haben wir uns dann umentschieden und eine Tilemap für das Leveldesign verwendet.

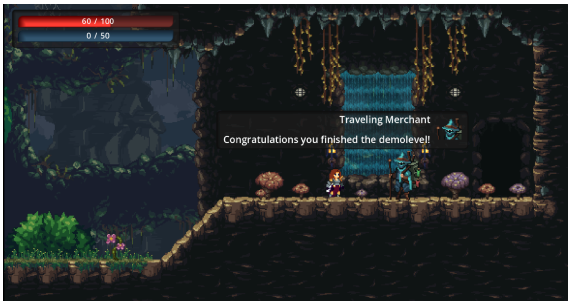
Das Tileset beinhaltet mehrere verschiedene kleinere Tilemaps, da wir uns für das Projekt ein großes Asset Pack des Künstlers “Szadi art” im Itch.io Sommersale gekauft haben. Zu den Standard Tilesets haben wir mehrere Sprites mit Hilfe der in Godot implementierten

Funktionen animiert, sodass wir im Ganzen eine große Menge an Assets zur Verfügung hatten.





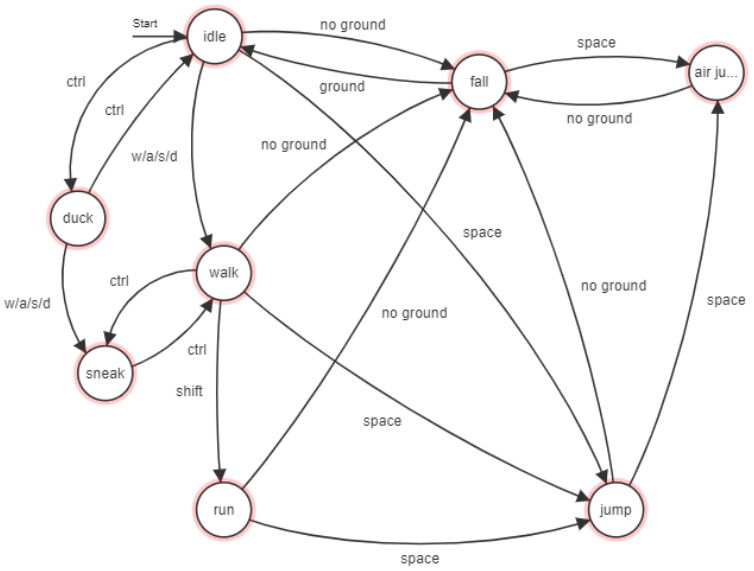




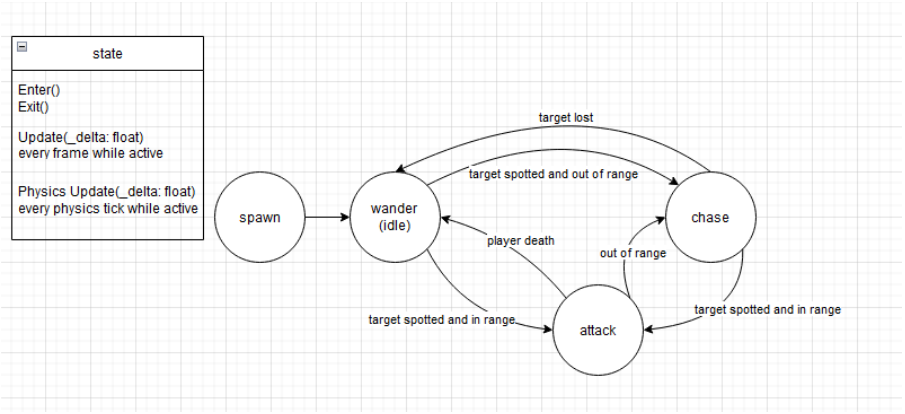
Steuerung

In das Design der State Machine für Spieler und Gegner haben wir vorab einige Zeit investiert, um herauszufinden, welche Funktionen wir für unser Spiel auf jeden Fall abdecken wollen.

Um dem Spieler möglichst viele Arten an die Hand geben, sich durch die Level unseres Sidescrollers zu bewegen, haben wir beschlossen, State Machines zu verwenden.



In der Planungsphase haben wir die Graphen für die States erstellt, ohne dass eine endgültige Entscheidung zu den verwendeten grafischen Assets gefällt war.
Wir haben daher so allgemein wie möglich geplant und versucht, die gewünschten States sinnvoll mit Kanten zu verbinden und bereits auch überlegt, welche Bedingungen für einen Wechsel gelten müssten.



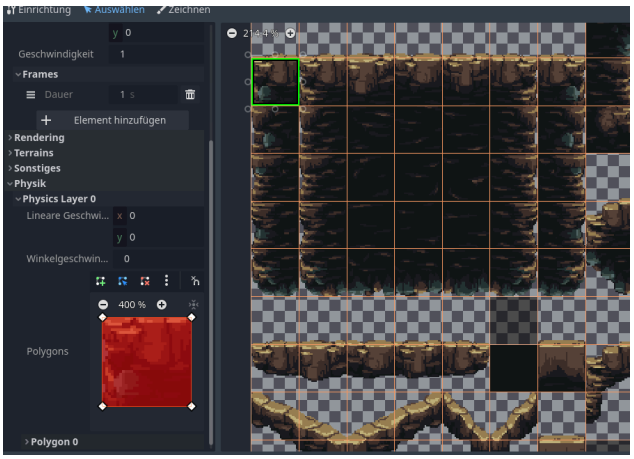
Im Gegensatz zu der komplexen Idee für die Player State Machine, war der Gedanke für die Gegner, die Enemy State Machine so simpel wie möglich zu halten.

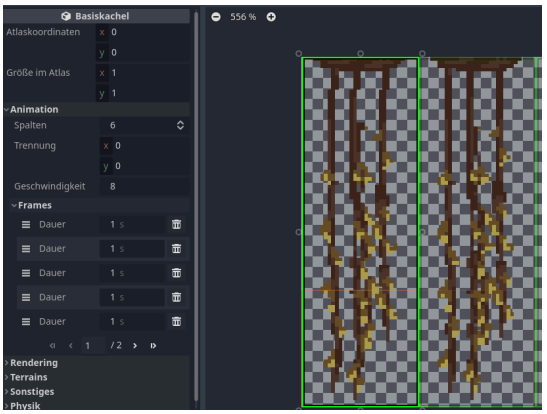
Umsetzung

Leveldesign

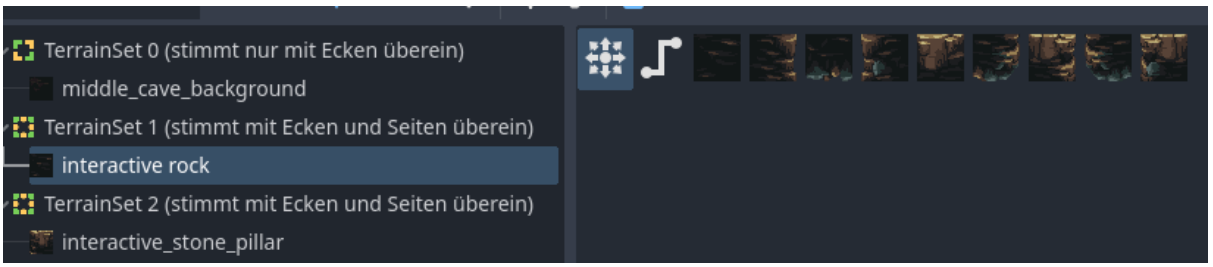
Tilemap und Audiodesign

Durch die große Menge an Assets konnten wir diese über verschiedene Methoden in unser Spiel integrieren.

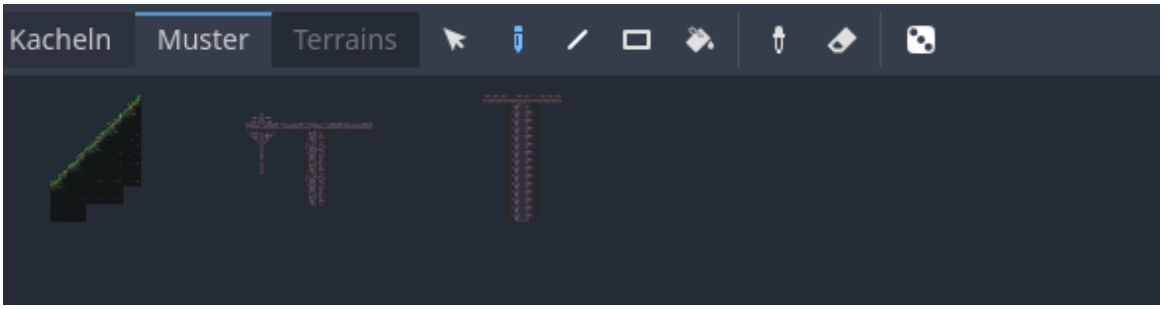




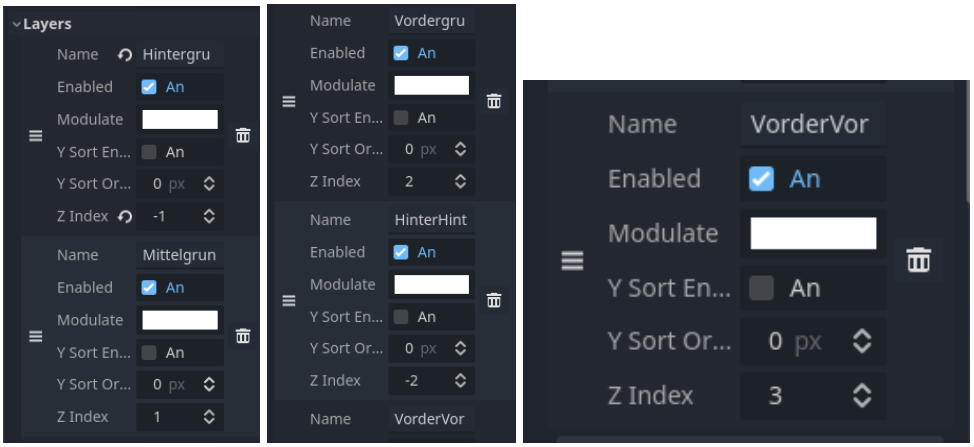
Einige Assets haben wir mithilfe des Auto-Tilings als Terrains implementiert, komplexere Formen wiederum ließen sich sinnvoller mit manuellem Einfügen realisieren.



Sich wiederholende Strukturen haben wir als Muster gespeichert, um sie nach Belieben öfter im Level zu platzieren.



Um der Welt mehr Tiefe zu geben, haben wir insgesamt 5 Layer erstellt. Der Spieler bewegt sich dabei auf der mittleren, sodass jeweils 2 Layer vor und hinter dem Player entstanden.



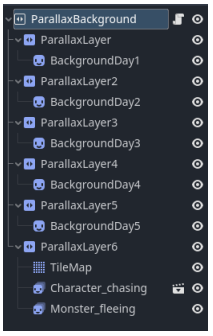
Das Level an sich ist unterteilt in 5 Bereiche: “Spawn-Area”, "Waterfall Crossing", “Cave”, “Dungeon” , "Demo End". Letzteres sollte als provisorischer Levelübergang dienen. Um diese Bereiche weitestgehend voneinander zu unterteilen, haben wir uns dazu entschieden, ihnen eigene Musikstücke zuzuweisen, die zu den jeweiligen Settings passen.

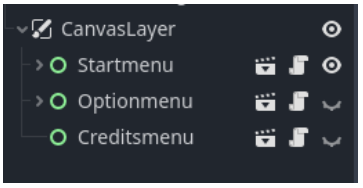
Zudem haben wir einzelnen “Objekten”, wie beispielsweise den Respawn-Punkten für den Spieler, Sound mithilfe von “Audio Area Szenen” verliehen, die durch das Betreten des CharacterBody2D des Characters ausgelöst werden.

Bei unserem Charakter haben wir zu fast allen States eigene SFX Sounds eingebunden, die ein besseres Spielerlebnis erzeugen. Dies ist aber im Moment auch eine der letzteren Baustellen unseres Projektes, da unser Charakter bisher noch keinen eigenen SFX Sound beim Laufen erzeugt, jedoch haben States wie Attack oder Dash bereits eigene Sounds. Zurzeit bauen wir dieses Feature noch weiter aus.

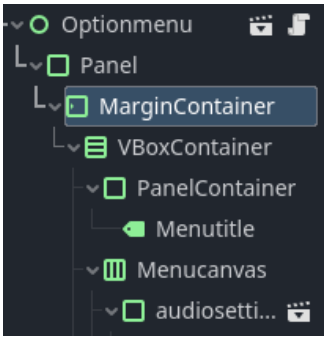
Main Screen

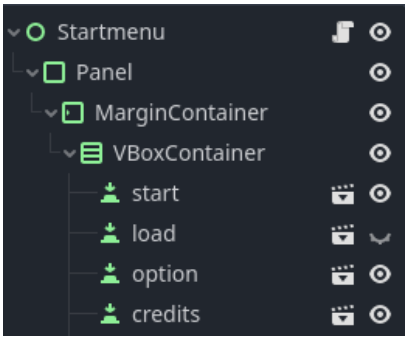
Der Mainscreen besteht aus einem Parallax Background, in dem mehrere Parallaxlayer liegen, die Node2D Knoten bzw. einen Sprite2D-Knoten mit dem Charakter enthalten, die Parallax Layer sind unterschiedlich schnell animiert.
In einem Canvas-Layer befinden sich verschiedene Menüs, darunter das Hauptmenü und ein Optionen-Menü.

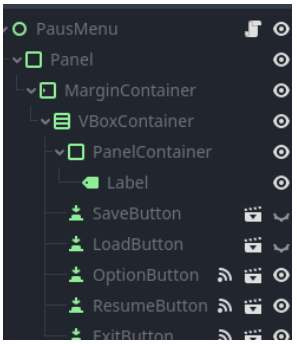




Menüs



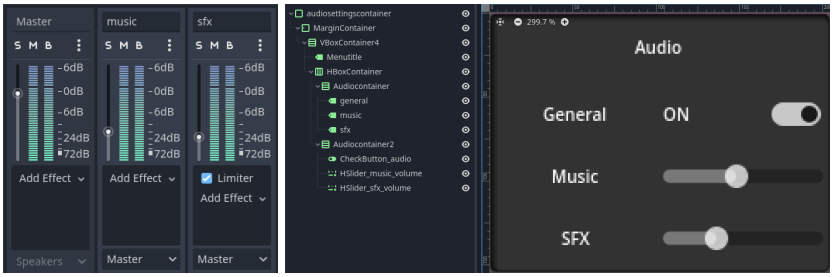




Die einzelnen Menüs bestehen aus mehreren verschiedenen Control Nodes und sind im Aufbau fast gleich:

- Control Node als Anker
- Panel Container
- Margin Container
- Vbox,Hbox
- Label, RichTextLabel
- Buttons, CheckButton
- Slider

Zu erwähnen ist noch, dass Buttons, HSlider und dergleichen als User-Schnittstelle dienen, die z.B. einen Szenenwechsel ins Spiel ermöglichen oder den AudioBus mithilfe von Signalen manipulieren.




```
func _on_option_button_pressed():
    self.hide()
    if get_parent():
        var option_menu =get_parent().find_child("Optionmenu")
        if option_menu:
            option_menu.show()
```

Audiocontroller

Der Audiocontroller ist ein Script, das sich mit der dynamischen Erstellung der Soundscape befasst. Er besteht aus mehreren Funktionen zum Abspielen unterschiedlicher Audiodateien (.wav für SFX, .mp3 für Musik) und deren Zuweisung zum korrekten Audiobus.
So können wir grundlegend mit dem Dateipfad und dem Audiotype den entsprechenden AudioStreamPlayer erzeugen und diesen nach Ende der Wiedergabe freigeben. Eine der vorhandenen Funktionen ist unter anderem der Crossfade Player der einen dynamischen übergang der Musikstücke ermöglicht

```
func _on_audio_change_request(filename: String, audioplayer: String, loop:bool=false):
    match [audioplayer]:
        ["MUSIC"]:
            var music_track = "%s%s.mp3" % [audio_path, filename]
            create_cross_fade_player(music_track)
            fading = true
        ["SFX"]:
            var sfx_track = "%s%s.wav" % [sfx_path, filename]
            play_sfx(sfx_track, loop)
            pass
        ["VOICE"]:
            #play_voice(filename)
            pass
```

```
func _on_sfx_finished()->void:
	for every_sfx_player in sfx_player_arr:
		if !every_sfx_player.playing:
			remove_child(every_sfx_player)
			sfx_player_arr.erase(every_sfx_player)
			every_sfx_player.queue_free()
```

```
func play_sfx(sfx_track: String, loop: bool = false):
#	print(sfx_track)
	sfx_player = AudioStreamPlayer.new()
	sfx_player.bus = "sfx"
	add_child(sfx_player)
	sfx_player.stream = load(sfx_track)
	if !loop:
		sfx_player.finished.connect(_on_sfx_finished)
```

State Machines

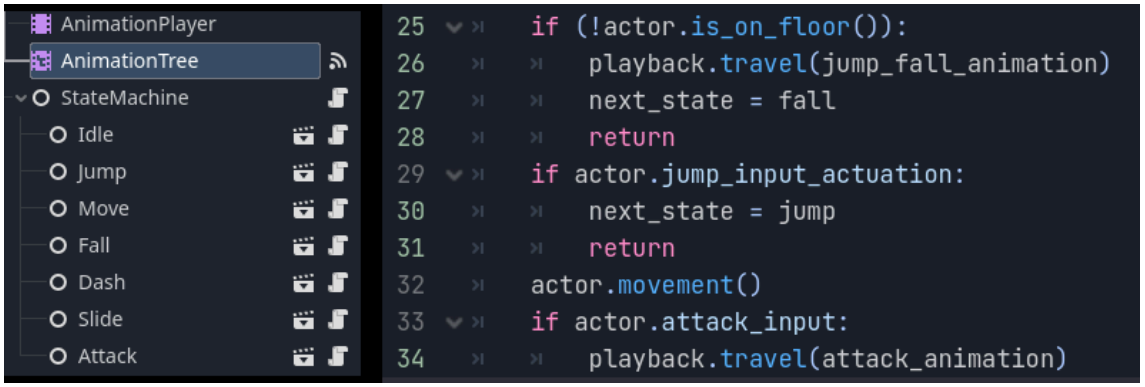
Sowohl für States, als auch für die State Machines selbst existieren abstrakte Klassen in Script-Form, die die wichtigsten Grundlagen an die Player Statemachine und ihre States, respektive die der Gegner vererben.

```
18 func change_state(new_state:State):
19 	if current_state != null:
20 		current_state.on_exit()
21 		current_state.next_state = null
22 	new_state.previous_state = current_state
23 	current_state = new_state
24 	current_state.on_enter()
```

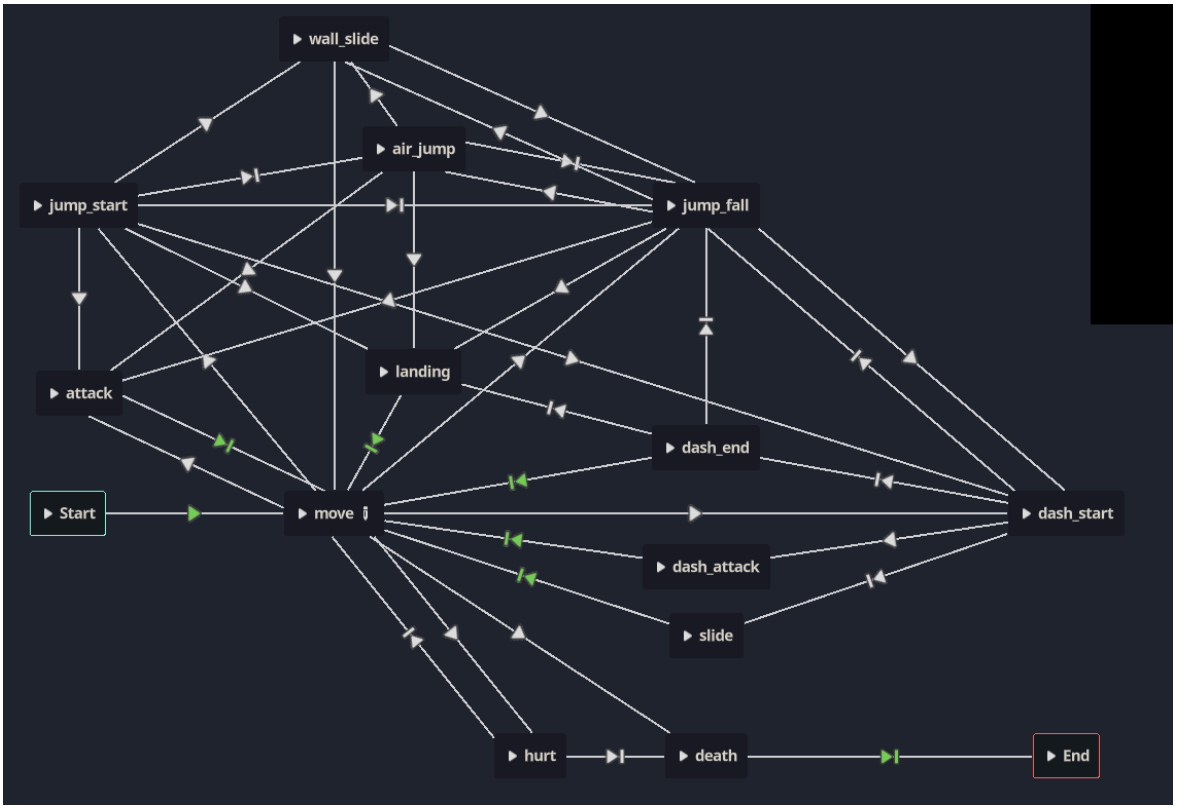
```
6
7  var actor:CharacterBody2D
8  var next_state:State = null
9  var previous_state:State = null
10 var playback:AnimationNodeStateMachinePlayback
11 var animation:AnimationPlayer
12
```

Player State Machine

Die entstandene State Machine für den Spieler weicht aufgrund der ausgewählten grafischen Assets leicht von der ursprünglichen Idee ab. Jedoch sind wir bei der Umsetzung so nah am Plan geblieben, wie es möglich war.
In dieser Version der Statemachine leisten die einzelnen States einen großteil der Aufgaben und die State Machine an sich hat relativ wenig mit den einzelnen Abläufen des Steuerung zu tun.
Dieser Ansatz wurde gewählt, um dem Spieler in jedem Zustand die größtmögliche Kontrolle zu gewähren und die Input-Abfragen in den jeweiligen States demnach unterschiedlich zu priorisieren.



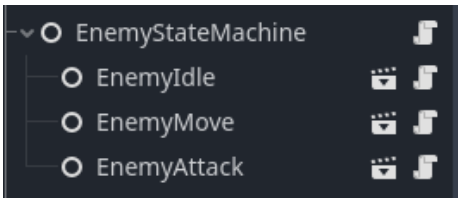
In der Spieler-Version der State Machine müssen zum aktuellen Stand die States alle sowohl ihre Vorgänger, als auch potentielle nachfolgende States kennen.

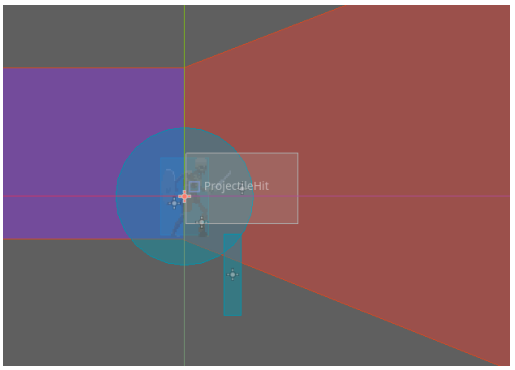


Animation Tree - AnimationNodeStateMachine

Enemy State Machine

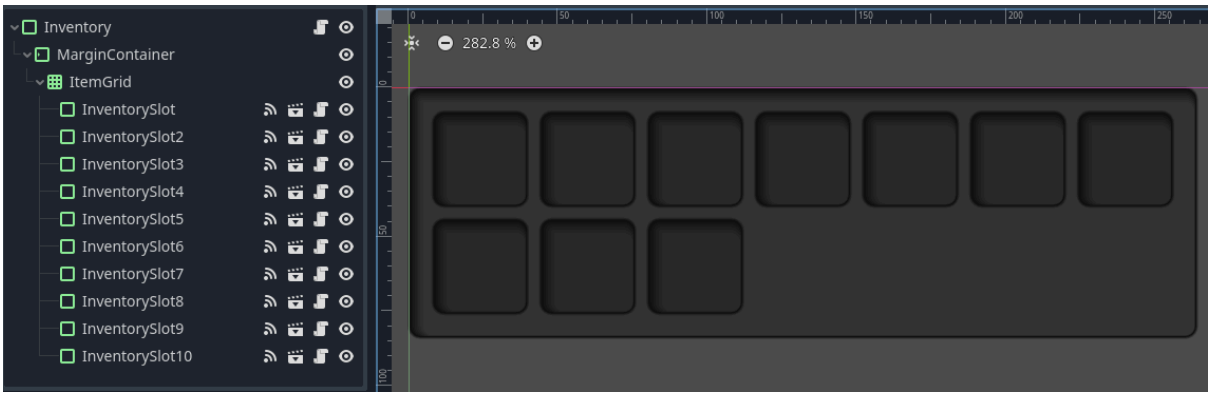
Die Enemy State Machine entstand eine ganze Zeit später als die des Spielers und löst ihre State-Wechsel smarter, indem sie Signale benutzt, um einen Übergang zu vermitteln.
Ein weiterer Unterschied besteht darin,dass die Statemachine einen Großteil der Entscheidungsfindung übernimmt und die einzelnen States lediglich zu spezifischen Situationen einen Beitrag liefern.





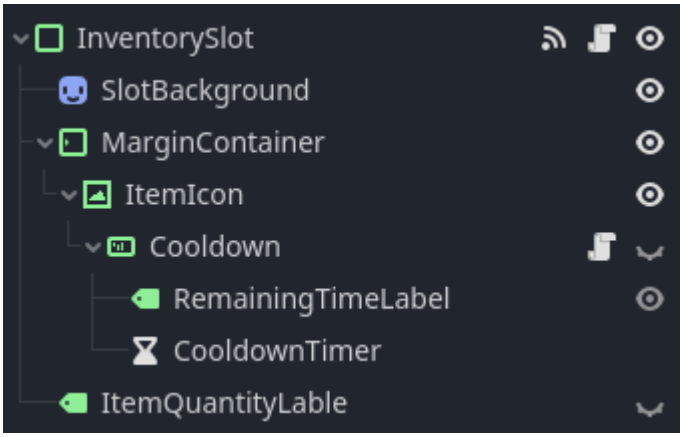
Inventory

Inventory Interface



- Das Inventory Interface besteht aus einem oder mehrerer:
- Panel Container um das Inventarfeld optisch etwas von dem Level abzuheben.
- Margin Container um einen Rand zwischen den Inventory Slots und dem Rahmen des Inventar Fensters zu halten.
- Grid Container, welcher zur Anordnung, aber auch Ansteuerung der Inventory Slots fungiert.
- Inventory Slots, welche den Gegenstand beinhalten, der in dem spezifischen Slot liegen soll.

Inventory Slots



Die Inventory Slots selbst bestehen wiederum aus einem oder mehreren:

- Panel und Margin Container aus den gleichen Gründen wie im Inventory Interface.
- Sprite2D um Hintergründe festzulegen, die in der Hotbar und dem Equipment Inventory zur Visualisierung des passenden Item Typs für den Slot genutzt wird.
- Label und TexturRect um das Item im Inventory Slot visuell darzustellen.

Hotbar

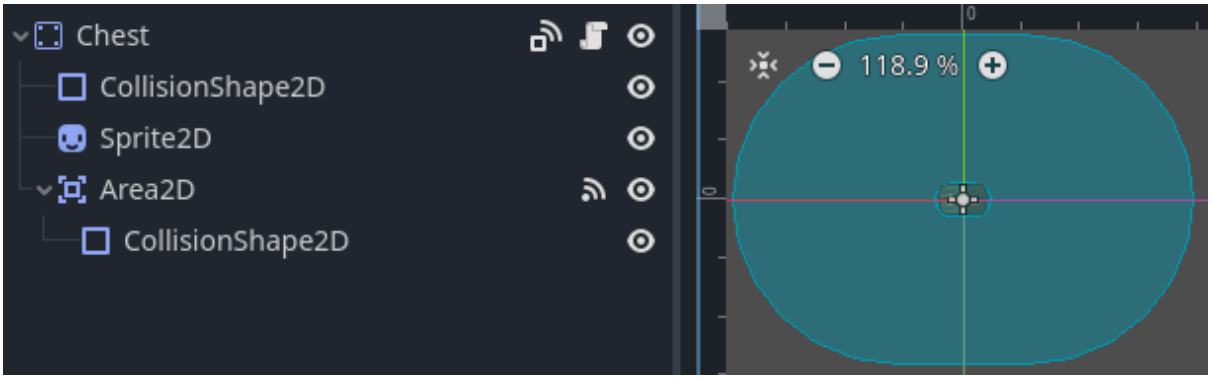
Der Inhalt des Inventars kann innerhalb des Inventory Interface problemlos verschoben werden. Lediglich beim Verschieben außerhalb des Inventory Interface gab es Komplikationen, wodurch die Hotbar als gespiegelte Version sowohl links vom eigentlichen

Inventar, als auch am unteren Bildschirmrand angezeigt wird. Mit der Hot Bar am unteren Bildschirmrand kann und soll nicht interagiert werden. Diese ist auch nicht sichtbar, wenn das Inventar geöffnet ist. In der Hotbar selbst können nur Resourcen vom typ ConsumableItems abgelegt werden.

Equipment Inventory

Ein Ausrüstungsgegenstand der Ressource Headwear kann lediglich im Inventory und im Equipment Inventory, Slot 0 (gekennzeichnet mit einem blassen Helm) angelegt werden. Dies gilt Analog für alle anderen Rüstungs-Resourcen.

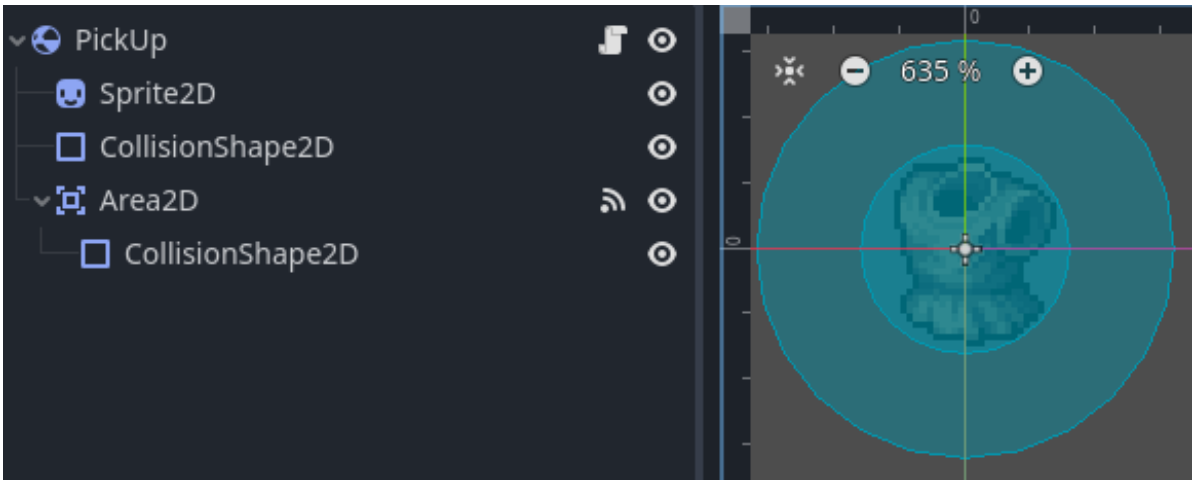
Chest



Ein Truhen-System ist auch vorhanden. Um eine Truhe zu öffnen, muss die Kiste eine Kollision mit dem Player erkannt haben (große Collision Shape) und der Cursor muss zusätzlich über der Kiste positioniert werden. (kleine Collision Shape). Sind beide Bedingungen erfüllt, so kann der Spieler mit der Taste “E” die gewählte Kiste öffnen. Verlässt er den Truhen-Bereich, so schließt sich die Truhe. Die Truhe verwendet hierbei die gleiche Inventory-Szene wie der Spieler selbst. Somit ist ein Transfer von Truhe zu Inventar und umgekehrt problemlos möglich.

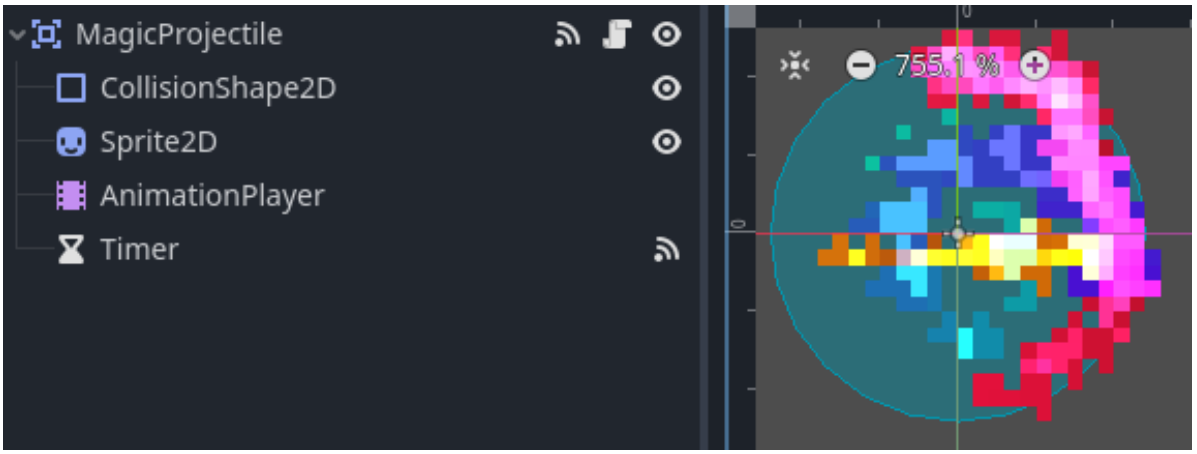
Item System

Pickups



Item Pickups bestehen aus zwei Collision Shapes, eine kleine zum Prüfen auf Gelände, eine größere zum Einsammeln des Itemdrops. Aktuell entstehen Itemdrops nur, wenn der Spiele einen Gegenstand aus dem Inventarfenster fallen lässt. Die Drops sind jedoch auch als Loot-Drop verwendbar, beispielsweise wenn ein Gegner besiegt wurde.

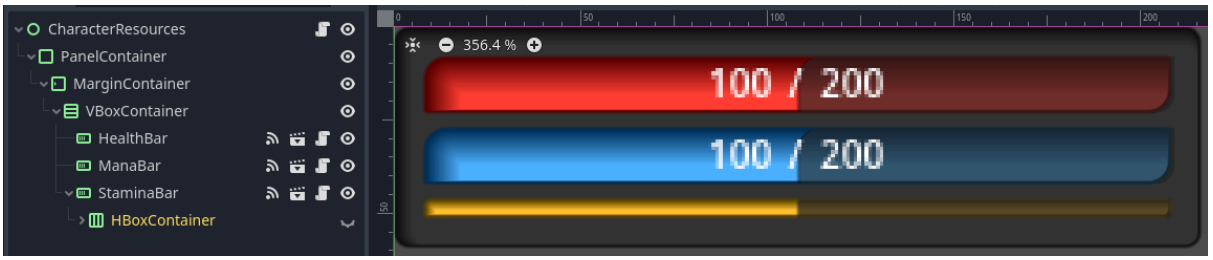
Skills



Skills wurden als Resource Consumables entworfen, die sich nicht verbrauchen, jedoch Mana benötigen, um gewirkt zu werden. Diese despawnen nach einer einstellbaren Zeit, oder wenn sie ein Objekt (TileMap, Chest, Enemy, etc.) treffen. Die passende Animation

muss bei der Erstellung des Projektils als String mitgegeben werden, sonst wird die default Animation in AnimationPlayer abgespielt. Cooldowns wurden angedacht, aber letztlich noch nicht umgesetzt.

Resource Interface



Das Resource Interface ist mit mehreren Progress Bars erstellt. Der eigentliche Wert der Bars entspricht hierbei aus optischen Gründen dem fehlenden Leben/Mana/Stamina des Spielers und die Bar steigt von rechts nach links. Die Werte wurden entsprechend umgerechnet. Stamina wurde nicht implementiert, aber die Möglichkeit dazu besteht.

Probleme und Lerneffekte

State Machines

Da die Player State Machine zuerst entwickelt wurde, entstand eine komplexe, ineinander verwobene Struktur, die sich im Nachhinein als keine schlaue Lösung herausgestellt hat. Die States müssen sich gegenseitig kennen, um voneinander aufgerufen werden zu können.

IdleState

Damit verbunden haben sich häufiger Probleme in den Animationen widergespiegelt. Unter anderem traten während der Umsetzung der ersten States immer wieder rekursive Verbindungen im Animation Tree auf und es ist auch in der Abgabeversion noch möglich, dass der Charakter unter bestimmten Bedingungen in einem State feststeckt. So sorgte beispielsweise das Erleiden eines Treffers durch Gegner und den Start der Dash-Animation im selben Moment für einen Endlos-Dash.

Es gab einen Versuch, die State Machine des Spielers vor dem Abgabetermin noch zu modernisieren, aber die dabei entstandenen Probleme zu beheben, hätte die Einhaltung des Abgabetermins gefährdet und die Gruppe hat einheitlich beschlossen es bei der bestehenden State Machine zu belassen.

Audiocontroller

Der Audiocontroller entstand ca. in der Mitte des Entwicklungsprozesses und war zunächst lediglich für das Verwalten der Musikstücke gedacht, da wir uns noch nicht sicher waren, inwiefern wir gut gelungene SFX umsetzen könnten. Da die einzelnen AudioStreamPlayer allesamt unterschiedlichste Einstellungen und Features unterstützen, hat sich das ganze doch als eine etwas größere Herausforderung entpuppt.
So hat sich der Controller im Laufe des Projektes mehrfach verändert und gilt bis zu diesem Zeitpunkt in Entwicklung.
Beispielsweise hatten wir Probleme mit den einzelnen Audioformaten unserer Soundscape, da sie in Godot verschiedene Attribute besitzen. Ebenso hatten wir mit dem dynamischen Erstellen der einzelnen AudioStreamPlayer Probleme, da diese zur Laufzeit längere Zeit nicht richtig freigegeben wurden. Im Moment besteht noch ein Problem mit den neu hinzugefügten Spawns, da das aktuelle Musikstück weitergespielt wird, auch wenn dieses zu einem anderen Gebiet gehört und der Bewegungs-Animation unseres Charakters fehlt noch Audiobegleitung.

Im Nachhinein wäre es teilweise wesentlich einfacher gewesen, bestimmte Sounds in den für die Animationen zuständigen Nodes zu integrieren, anstatt diese über den Audiocontroller zu steuern.

Nichtsdestotrotz ist der Audiocontroller zum jetzigen Zeitpunkt extrem flexibel und steuert die in unserem Projekt vorkommende Soundscape.

Der Lerneffekt hierbei bestand darin, sich mit den einzelnen Nodes und der gegebenen Dokumentation intensiver auseinanderzusetzen, um für spezielle Probleme, die am Anfang nicht ersichtlich waren, bessere Lösungsansätze zu finden..

Quellenangaben für Assets

Soundscape:

Alexander Nakarada (CreatorChords)
https://www.free-stock-music.com/artist.alexander-nakarada.html
Adventure, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-adventure.html
Halloween Theme 1, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-halloween-theme-1.html
Medieval Loop One, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-medieval-loop-one.html
Soft Interlude, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-soft-interlude.html
Superepic, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-superepic.html
The Great Battle, CC BY 4.0
https://www.free-stock-music.com/alexander-nakarada-the-great-battle.html
jeweils letzter Zugriff: 05.10.2023

TomMusic
Free Fantasy SFX and Music Bundle, lizenzgebührenfrei
unter: https://tommusic.itch.io/free-fantasy-sfx-and-music-bundle
"*Royalty-free! You can use this in your projects, commercial or otherwise, credit is not mandatory but appreciated. The asset is not to be up for resale or redistribution.*"
letzter Zugriff: 05.10.2023

Grafik:

Tilesets

Szadi art. - https://itch.io/profile/szadiart
Sidescroller-Worlds 1-7, Sidescroller-.Village 1-2, Sidescroller-Castlepack
https://szadiart.itch.io/sidescroll-worlds-set1
https://szadiart.itch.io/sidescroll-worlds-set3
https://szadiart.itch.io/sidescroll-worlds-set2
https://szadiart.itch.io/sidescroll-worlds-set4
https://szadiart.itch.io/sidescroll-worlds-set5
https://szadiart.itch.io/sidescroll-worlds-set6
https://szadiart.itch.io/sidescroll-worlds-set7
https://szadiart.itch.io/sidescroll-worlds-castle-pack
https://szadiart.itch.io/sidescroll-worlds-village-pack1
https://szadiart.itch.io/sidescroll-worlds-village-pack2
https://szadiart.itch.io/sidescroll-game-player

>>> License for Everyone.

You can use the Licensed Asset to create Public domain and free to use, personal or commercial.

> *You can use the Licensed Asset:*
> *(a) edit and modify, cut and change the asset used for commercial purposes;*
> *(b) sell works created with the assets;*
> *(c) publish on websites related to graphics, games and similar.*
> *>>> A Licence does not allow the Purchaser to:*
> *(d) Use the Licensed Asset or Derivative Works in a logo, trademark or service mark*
> *(e) Resell it, original or modified;*

Credit is not required but appreciated.

letzter Zugrif auf die oberhalb aufgelisteten Links: 05.10.2023

Sprites

Clembod - https://itch.io/profile/clembod
Warrior-Free Animation set V1.3
unter: https://clembod.itch.io/warrior-free-animation-set
"*You can use this asset for personal and commercial purpose, you can modify this object to your needs.Credit is not required but would be appreciated*
You can NOT redistribute or resell it."
letzter Zugriff: 05.10.2023

Luizmelo - https://itch.io/profile/luizmelo
unter: https://luizmelo.itch.io/monsters-creatures-fantasy
"This package can be used in commercial and non-commercial projects - CC0 (creative commons zero)"
letzter Zugriff: 05.10.2023

XYEzawr - https://itch.io/profile/xyezawr
unter: https://xyezawr.itch.io/gif-free-pixel-effects-pack-12-mini-magick-shoots-3
"*LICENCE: This asset pack can be used in both free and commercial projects. You can modify it to suit your own needs. Credit is not necessary, but appreciated. You may not redistribute it or resell it.*"
letzter Zugriff: 05.10.2023

Clockwork Raven - https://clockworkraven.itch.io
Lizenz: https://drive.google.com/file/d/1DWGfcVE5nXsAkaPgqfGqf0lwesv6ug3N/view

https://clockworkraven.itch.io/raven-fantasy-hd-pixel-art
https://clockworkraven.itch.io/raven-fantasy-pixel-art-rpg-icons-crops-food-and-drinks
https://clockworkraven.itch.io/raven-fantasy-hd-rpg-icons-pixel-art-icons-textures-and-sprites-armor
https://clockworkraven.itch.io/raven-fantasy-hd-rpg-icons-pixel-art-icons-textures-and-sprites-helmets-and-head
https://clockworkraven.itch.io/raven-fantasy-hd-rpg-icons-pixel-art-icons-textures-and-sprites-boots-and-fo
https://clockworkraven.itch.io/raven-fantasy-hd-rpg-icons-pixel-art-icons-textures-and-sprites-gloves-and-g
letzter Zugrif auf die oberhalb aufgelisteten Links: 05.10.2023

joraxus
unter: https://inspired-goblin.itch.io/chest-mimic
"This pack can be used in free and commercial projects. You can modify the assets as you need. Credit is not necessary, but always appreciated. You may not repackage, redistribute or resell the assets, no matter how much they are modified."
letzter Zugriff: 05.10.2023